

SISTEMAS OPERACIONAIS

SEMANA 1 - AULA 1

Gilberto Falco Netto

Tecnólogo em Análise e Desenvolvimento de Sistemas,
Faculdade Anhanguera - Jundiaí/SP

Padrão EMV: O que é?

EMV é a sigla para Europay, Mastercard e Visa, as empresas que criaram o padrão

- Padrão de segurança para transações com cartões de pagamento.
- Utiliza chips em vez de tarjas magnéticas.
- Desenvolvido para reduzir fraudes e aumentar a segurança.



Principais Características do EMV

- **Chip Embutido:** Microprocessador que armazena informações criptografadas.
- **Criptografia:** Proteção de dados durante as transações.
- **Autenticação:** Verificação da identidade do usuário (PIN, contactless).
- **Redução de Fraudes:** Geração de códigos dinâmicos e autenticação.
- **Padrão Global:** Adotado por bancos e instituições financeiras em todo o mundo.

Como Funciona o EMV?

- Cartão EMV é inserido em terminal compatível.
- Chip e terminal se comunicam.
- Chip gera código criptografado único para a transação.
- Código é enviado ao banco emissor para autorização.
- Banco verifica e autoriza a transação.

Benefícios do EMV

- Maior segurança para transações com cartões.
- Redução de fraudes e clonagem de cartões.
- Facilidade de uso com a tecnologia contactless.
- Interoperabilidade global dos cartões.

EMV em Outros Cartões Inteligentes

- Presente em cartões de crédito e débito.
- Utilizado em cartões de transporte público.
- Implementado em cartões de identificação.

O que é um Processo?

Definição

Um processo é uma instância em execução de um programa. Ele representa a atividade de um programa, independentemente de outros programas estarem em execução.

Exemplo

Ao abrir um editor de texto, mesmo sem outros programas em execução, um novo processo é criado.

Outro exemplo: Ao executar um vídeo no player de mídia, um processo é iniciado para gerenciar a reprodução do vídeo.

Pseudoparalelismo: Gerenciando Múltiplos Processos

Como Funciona?

Em sistemas com uma única CPU, o pseudoparalelismo permite a execução de múltiplos processos ao alternar rapidamente entre eles, criando a ilusão de execução simultânea.

Exemplo

Enquanto você assiste a um vídeo e recebe notificações em um sistema de uma única CPU, o sistema alterna entre os dois processos para dar a impressão de que ambos estão ocorrendo ao mesmo tempo.

Multiprogramação: Melhorando a Utilização da CPU

O que é?

A multiprogramação permite que vários processos coexistam na memória ao mesmo tempo, aumentando a utilização da CPU ao minimizar o tempo ocioso.

Exemplo

Ao imprimir um documento enquanto edita uma planilha, ambos os processos estão na memória e a CPU alterna entre eles para garantir que as duas tarefas sejam realizadas eficientemente.

Processo vs. Programa: Receita vs. Bolo

Programa

Um programa é um conjunto estático de instruções, como uma receita de bolo.

Processo

Um processo é a execução dinâmica do programa, semelhante ao bolo sendo preparado.

Exemplos

Programa: Arquivos como .exe, .py, .java

Processo: Uma instância em execução de um programa

Exemplo: O arquivo de instalação do Word é um programa. Ao abrir o Word para editar um documento, você cria um processo.

Analogia do Bolo: Componentes de um Processo

Componentes

- Ingredientes: Dados do processo (variáveis, entrada do usuário)
- Receita: Programa em execução (sequência de instruções)
- Cozinheiro: CPU executando o processo (realiza as instruções)

Exemplo

Ao rodar um programa de edição de imagem:

- Ingredientes: Imagem carregada e ajustes aplicados
- Receita: Código do editor de imagem
- Cozinheiro: CPU processando as edições

Interrupção e Retorno: Troca de Contexto

Interrupção

Evento que pausa a execução do processo atual, podendo ser gerado por hardware ou software.

Troca de Contexto

Quando ocorre uma interrupção, a CPU salva o estado do processo em execução e carrega o estado de outro processo para continuar a execução.

Exemplo

Ao receber uma ligação em um smartphone enquanto assiste a um vídeo, o sistema pausa a reprodução (interrupção) e alterna para o aplicativo de chamadas (troca de contexto).

Processo: Definição Completa e Componentes

Definição

Um processo é uma unidade de execução que possui recursos alocados pelo sistema operacional.

Componentes Principais

- Espaço de endereçamento (código, dados e pilha)
- Arquivos abertos
- Estado de execução (registradores, contador de programa)

Exemplo

Ao abrir um navegador, ele cria processos para renderizar páginas, gerenciar abas e controlar plugins, cada um com seus próprios recursos e estados.

Criação de Processos: Nascendo para Executar

Mecanismos de Criação

O sistema operacional cria processos usando chamadas de sistema, como `fork()` e `exec()`.

Pai e Filho

Um processo pai pode criar processos filhos, que herdam parte dos seus recursos.

Hierarquia de Processos no Unix

Processo Pai e Filho

No Unix, quando um processo cria outro, eles ficam associados.

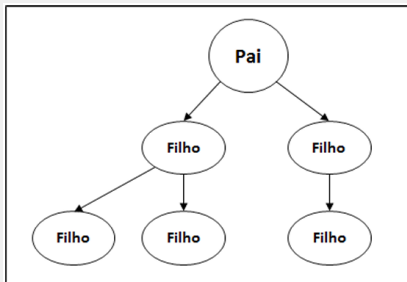
Exemplo

Um shell (processo pai) cria um editor de texto (processo filho).

Hierarquia de Processos no Unix

Árvore de Processos

O processo filho pode criar outros processos, formando uma hierarquia em árvore.



Hierarquia de Processos no Unix

Grupos no Unix

Um processo pai, seus filhos e descendentes formam um grupo de processos.

Exemplo

Um servidor web (pai) cria múltiplos processos para atender requisições (filhos).

Hierarquia de Processos no Unix

Sinais para Grupos

Sinais enviados pelo teclado (ex: CTRL+C) são entregues a todos os processos do grupo.

Atenção!

Quando um processo pai é terminado, todos os filhos vinculados a ele também são terminados.

Estados de um Processo: O Ciclo de Vida

Principais Estados

- **Pronto:** Aguardando para ser executado pela CPU
- **Executando:** Em execução ativa na CPU
- **Esperando:** Pausado, aguardando um evento externo (E/S, sincronização)
- **Suspenso:** Temporariamente removido da memória principal

Estados de um Processo: O Ciclo de Vida

Exemplo

Ao baixar um arquivo da internet:

- Pronto: O processo de download está aguardando tempo de CPU
- Executando: O processo está transferindo os dados
- Esperando: Pausa ao aguardar resposta do servidor
- Suspenso: Download pausado manualmente



Threads: Introdução

O que são Threads?

Threads são fluxos de controle dentro de um processo, também chamados de processos leves.

Objetivo

Reduzir o tempo gasto na criação, eliminação e troca de contexto de processos.

Por que usar Threads?

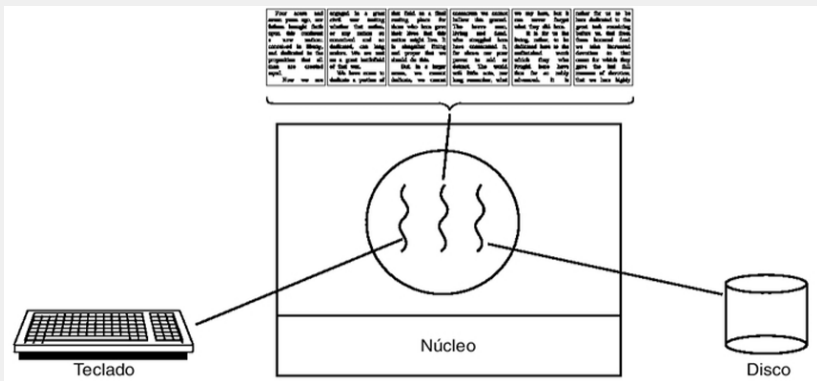
- **Concorrência:** Executar múltiplas atividades em paralelo.
- **Eficiência:** Criação e destruição mais rápidas que processos.
- **Desempenho:** Acelerar aplicações com muitas operações de E/S.

Threads no Processador de Texto

Cenário: Escrever um livro com várias correções e formatações.

- **Thread 1:** Formatação do texto.
- **Thread 2:** Interação com o usuário (teclado).
- **Thread 3:** Backup dos dados em disco.

Exemplo: Processador de Texto



Exemplo: Navegador Web

Concorrência

Enquanto um thread carrega imagens e textos, outro recupera dados da rede.

Implementação de Processos: Quadro de Processos

Quadro de Processos

O SO mantém informações sobre o estado, contador de programa, pilha, memória, arquivos, etc.

Quadro de Processos: Detalhes

- **Gerenciamento de Processos:** Registradores, contador de programa, estado, prioridade.
- **Gerenciamento de Memória:** Ponteiros para segmentos de código, dados e pilha.
- **Gerenciamento de Arquivos:** Diretório raiz, diretório de trabalho, descritores de arquivos.

Implementação de Threads: Tipos

- **Threads de Usuário:** Implementadas pela aplicação, SO não sabe da existência.
- **Threads do Núcleo:** Implementadas e gerenciadas pelo SO.
- **Threads Híbridos:** Implementadas tanto no espaço do usuário quanto no núcleo.

Threads de Usuário

Vantagens

- **Rápidas e Eficientes:** Troca de contexto rápida, sem chamadas ao SO.
- **Implementação Simples:** Bibliotecas no espaço do usuário.
- **Portabilidade:** Executam em diferentes plataformas sem modificações.

Threads de Usuário

Desvantagens

- **Bloqueio do Processo:** Chamadas de sistema bloqueantes afetam todas as threads.
- **Não Utilizam Multiprocessadores:** Não executam em paralelo em múltiplos núcleos.
- **Escalonamento Limitado:** Escalonamento feito pela biblioteca, menos eficiente.

Exemplo: Editor de Texto

Verificação ortográfica e formatação automática em segundo plano.

Threads do Núcleo

Vantagens

- **Gerenciamento pelo SO:** Escalonamento eficiente.
- **Utilização de Multiprocessadores:** Executam em paralelo em múltiplos núcleos.
- **Não Bloqueiam o Processo:** Chamadas de sistema bloqueantes afetam apenas a thread.

Threads do Núcleo

Desvantagens

- **Chamadas ao Sistema:** Troca de contexto mais lenta.
- **Implementação Complexa:** Dependência do SO.
- **Menos Portabilidade:** Dependência do sistema operacional.

Exemplo: Servidor Web

Atendimento de requisições de clientes simultaneamente.

Threads Híbridos

Vantagens

- **Flexibilidade:** Combinação de threads de usuário e do núcleo.
- **Escalonamento Adaptável:** Biblioteca e SO.
- **Utilização de Multiprocessadores:** Executam em paralelo em múltiplos núcleos.

Threads Híbridos

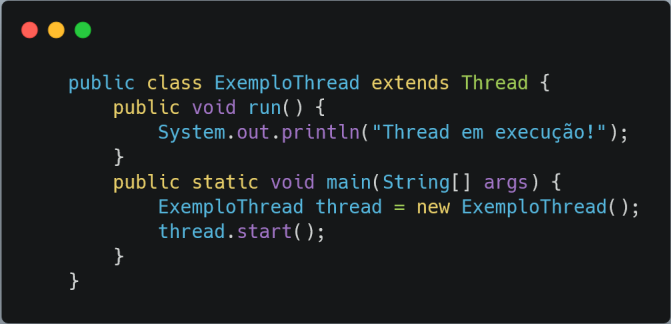
Desvantagens

- **Complexidade de Implementação:** Maior complexidade.
- **Gerenciamento Mais Complexo:** Biblioteca e SO.
- **Possíveis Problemas de Sincronização:** Necessidade de coordenação.

Exemplo: Aplicativo de Banco de Dados

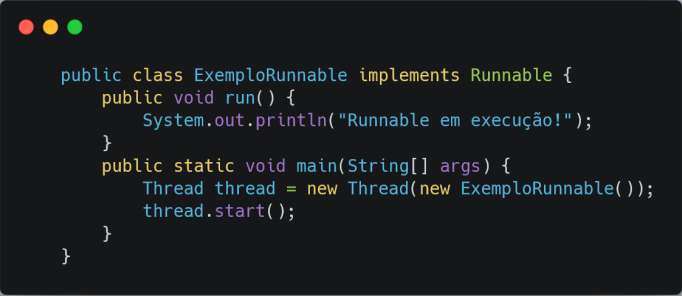
Consultas complexas com threads de usuário e do núcleo.

Threads em Java: Exemplo de Código



```
public class ExemploThread extends Thread {  
    public void run() {  
        System.out.println("Thread em execução!");  
    }  
    public static void main(String[] args) {  
        ExemploThread thread = new ExemploThread();  
        thread.start();  
    }  
}
```

Threads em Java: Interface Runnable



```
public class ExemploRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable em execução!");  
    }  
    public static void main(String[] args) {  
        Thread thread = new Thread(new ExemploRunnable());  
        thread.start();  
    }  
}
```

Concorrência em Threads: Sincronização

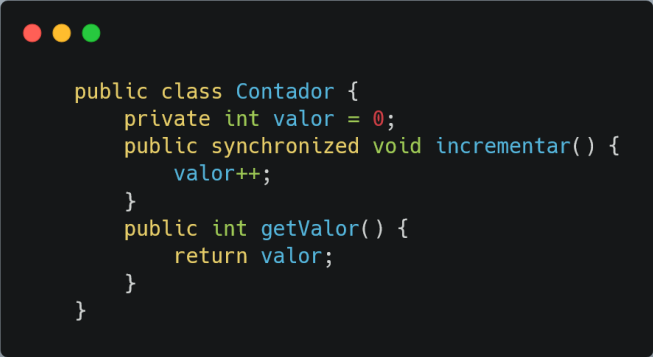
Problema

Múltiplos threads acessando recursos compartilhados podem causar inconsistências.

Solução

Utilizar sincronização (synchronized, locks) para garantir acesso seguro.

Exemplo de Sincronização em Java



```
public class Contador {  
    private int valor = 0;  
    public synchronized void incrementar() {  
        valor++;  
    }  
    public int getValor() {  
        return valor;  
    }  
}
```

Conclusão: Threads e Concorrência

Threads

Permitem a execução de múltiplas tarefas dentro de um processo, melhorando o desempenho e a eficiência.

Sincronização

Essencial para garantir a integridade dos dados em ambientes concorrentes.